

Documento Técnico

Proyecto Dino App

Nombre del Proyecto:	Aplicación de Gestión de Dinosaurios
Nombre:	Jessica Ayelen Vazquez Lopez
Profesor:	Adolfo Centeno Tellez
Materia:	Principios de diseño de software
Fecha:	08/Mayo/2026

Desarrollo con Flutter, arquitectura limpia y patrones de diseño

Índice

Introducción	2
Objetivo General	2
Objetivos Específicos	2
Análisis del Sistema	2
Requerimientos Funcionales	2
Requerimientos No Funcionales	3
Arquitectura y Stack Tecnológico	3
Tecnologías Utilizadas	3
Arquitectura de Carpetas	3
Patrones de Diseño	4
Singleton	4
Factory	4
Builder	5
Facade	6
Repository	6
Adapter	7
Decorator	7
Conclusiones y Trabajo Futuro	8
Retos Superados	8
Mejoras Futuras	8

Introducción

Esta aplicación móvil se creó para resolver el problema de gestionar datos de dinosaurios de forma organizada y moderna. En lugar de trabajar con estructuras fragmentadas o lógica mezclada, la aplicación utiliza una arquitectura limpia que separa claramente responsabilidades en servicios, modelos, providers y pantallas. El objetivo fue diseñar una solución que permita crear, visualizar, comentar y reaccionar a dinosaurios, con una base robusta que facilite mantenimiento y ampliaciones.

Objetivo General

Construir una aplicación móvil funcional y profesional para la gestión de dinosaurios, aplicando patrones de diseño que aseguren código reutilizable, fácil de mantener y extensible, mientras se emplea Flutter para ofrecer una experiencia uniforme en dispositivos móviles.

Objetivos Específicos

- Implementar autenticación con JWT para proteger el acceso a la app.
- Desarrollar un sistema CRUD para dinosaurios.
- Agregar comentarios y reacciones por dinosaurio.
- Aplicar patrones de diseño creationales y estructurales para una arquitectura ordenada.
- Utilizar Provider para la gestión reactiva del estado.
- Diseñar una estructura de carpetas clara y modular.

Análisis del Sistema

Requerimientos Funcionales

- El usuario debe poder registrarse e iniciar sesión.
- El usuario debe poder listar dinosaurios existentes.
- El usuario debe poder crear nuevos dinosaurios.
- El usuario debe poder eliminar dinosaurios.
- El usuario debe poder agregar comentarios a un dinosaurio.
- El usuario debe poder reaccionar a dinosaurios y comentarios.

Requerimientos No Funcionales

- La aplicación debe ser responsiva en Android e iOS.
- El sistema debe manejar errores de red y mostrar mensajes claros.
- El rendimiento debe ser ágil en las operaciones comunes.
- La seguridad debe garantizar almacenamiento seguro de tokens JWT.
- El código debe ser modular y fácil de escalar.

Arquitectura y Stack Tecnológico

Tecnologías Utilizadas

- **Lenguaje:** Dart
- **Framework:** Flutter
- **Backend:** API REST propia
- **Gestión de estado:** Provider
- **Almacenamiento local:** SharedPreferences
- **Cliente HTTP:** Paquete `http`

Arquitectura de Carpetas

La aplicación está organizada en módulos claros:

- `core/` — patrones de diseño, gestión HTTP y facades.
- `models/` — entidades de dominio como `Dinosaurio` y `Comentario`.
- `repositories/` — interfaces que abstraen el acceso a datos.
- `services/` — servicios Singleton que encapsulan la lógica de negocio.
- `providers/` — administración de estado con `ChangeNotifier`.
- `screens/` — las vistas de la aplicación.

Esta organización demuestra disciplina en el desarrollo y facilita encontrar cada responsabilidad.

Patrones de Diseño

Singleton

Para qué sirve: Garantiza una sola instancia de una clase a lo largo de la aplicación, evitando instancias duplicadas y compartiendo estado global de forma controlada.

Dónde se usa:

- `services/auth_service.dart`
- `services/dino_service.dart`
- `core/facades/app_facade.dart`

Ejemplo real usado en la app:

```
1 class AuthService {
2   static final AuthService _instance = AuthService._internal();
3   factory AuthService() => _instance;
4   AuthService._internal();
5
6   Future<bool> login(String username, String password) async {
7     // ...
8   }
9 }
```

Qué hace en esa parte:

- Asegura que sólo exista una instancia de `AuthService`.
- Centraliza la lógica de autenticación.
- Evita inconsistencias al acceder a token y username.

Factory

Para qué sirve: Centraliza la creación de objetos complejos a partir de datos externos, especialmente JSON, realizando validaciones y conversiones.

Dónde se usa:

- `models/dinosaurio.dart`
- `models/comentario.dart`
- `models/auth_response.dart`

Ejemplo real usado en la app:

```
1 factory Dinosaurio.fromJson(Map<String, dynamic> json) {
2   return Dinosaurio(
3     id: json['id'] as int?,
4     nombre: json['nombre'] as String? ?? 'Sin nombre',
5     estaturaAprox: (json['estaturaAprox'] as num?)?.toDouble() ?? 0.0,
6     esperanzaVida: json['esperanzaVida'] as int? ?? 0,
7     alimentacion: json['alimentacion'] as String? ?? 'Desconocido',
8     imageUrl: json['imageUrl'] as String? ?? '',
```

```
9     likesCount: json['likesCount'] as int? ?? 0,  
10    encantaCount: json['encantaCount'] as int? ?? 0,  
11    divierteCount: json['divierteCount'] as int? ?? 0,  
12    asombrosoCount: json['asombrosoCount'] as int? ?? 0,  
13  );  
14 }
```

Qué hace en esa parte:

- Convierte la respuesta del API en un objeto `Dinosaurio`.
- Maneja valores nulos con defaults seguros.
- Facilita el uso de datos JSON en el resto de la app.

Builder

Para qué sirve: Permite construir objetos complejos paso a paso, manteniendo el código legible y validando campos antes de crear el objeto.

Dónde se usa:

- `models/dinosaurio.dart`
- `models/comentario.dart`

Ejemplo real usado en la app:

```
1 class DinosaurioBuilder {  
2   String _nombre = '';  
3   double _estaturaAprox = 0.0;  
4   int _esperanzaVida = 0;  
5   String _alimentacion = '';  
6   String _imagenUrl = '';  
7  
8   DinosaurioBuilder withNombre(String nombre) {  
9     _nombre = nombre;  
10    return this;  
11  }  
12  
13  Dinosaurio build() {  
14    if (_nombre.isEmpty) {  
15      throw ArgumentError('El nombre del dinosaurio es requerido');  
16    }  
17    return Dinosaurio(  
18      nombre: _nombre,  
19      estaturaAprox: _estaturaAprox,  
20      esperanzaVida: _esperanzaVida,  
21      alimentacion: _alimentacion,  
22      imagenUrl: _imagenUrl,  
23    );  
24  }  
25 }
```

Qué hace en esa parte:

- Permite crear un dinosaurio de forma fluida.
- Evita constructores con muchos parámetros posicionales.
- Valida que el nombre sea obligatorio.

Facade

Para qué sirve: Oculta la complejidad de múltiples servicios detrás de una única capa de acceso, facilitando el uso desde las pantallas.

Dónde se usa:

- `core/facades/app_facade.dart`

Ejemplo real usado en la app:

```
1 class AppFacade extends Facade {  
2   Future<List<Dinosaurio>> getAllDinosaurios() async {  
3     return _dinoService.getAll();  
4   }  
5  
6   Future<bool> createDinosaurio(Dinosaurio dinosaurio) async {  
7     return _dinoService.create(dinosaurio);  
8   }  
9 }
```

Qué hace en esa parte:

- Simplifica las llamadas de UI a través de un solo objeto global.
- Separa el consumo de servicios de la presentación.
- Mejora la mantenibilidad, porque la UI no necesita conocer cada servicio.

Repository

Para qué sirve: Define contratos para el acceso a datos, separando la lógica de persistencia de la lógica de negocio.

Dónde se usa:

- `repositories/repository_interfaces.dart`

Ejemplo real usado en la app:

```
1 abstract class IDinoRepository {  
2   Future<List<Dinosaurio>> getAll();  
3   Future<bool> create(Dinosaurio dinosaurio);  
4   Future<bool> delete(int dinoId);  
5   Future<bool> toggleReaction(int dinoId, String reactionType);  
6 }
```

Qué hace en esa parte:

- Permite cambiar la implementación de datos sin modificar consumidores.
- Facilita el uso de mocks en pruebas.
- Clarifica responsabilidades entre capa de datos y capa de negocio.

Adapter

Para qué sirve: Normaliza datos de una fuente externa para que el resto de la app pueda trabajarlos en un formato consistente.

Dónde se usa:

- `core/http/http_manager.dart`

Ejemplo real usado en la app:

```
1 static Future<HttpResponse<T>> get<T>(  
2   String endpoint, {  
3   required T Function(Map<String, dynamic>) parser,  
4   String? token,  
5   Map<String, String>? extraHeaders,  
6 }) async {  
7   final response = await http.get(  
8     Uri.parse('$_baseUrl$endpoint'),  
9     headers: _buildHeaders(token, extraHeaders),  
10  );  
11  return _handleResponse(response, parser);  
12 }
```

Qué hace en esa parte:

- Toma respuestas HTTP y las transforma a objetos Dart.
- Centraliza la construcción de encabezados y el manejo de excepciones.
- Permite usar un parser genérico para cualquier modelo.

Decorator

Para qué sirve: Agrega responsabilidades adicionales a objetos sin cambiar su implementación principal.

Dónde se usa:

- `core/patterns/decorator.dart`

Ejemplo real usado en la app:

```
1 class LoggingDecorator<T> extends Decorator<T> {  
2   Future<R> executeWithLogging<R>(  
3     String operationName,  
4     Future<R> Function() operation,  
5   ) async {  
6     final startTime = DateTime.now();  
7     print('      Iniciando: $operationName');  
8     final result = await operation();  
9     print('    Completado: $operationName');  
10    return result;  
11  }  
12 }
```

Qué hace en esa parte:

- Permite agregar logging sin modificar la clase base.
- Facilita medir tiempos de ejecución.
- Mejora la capacidad de diagnóstico de la app.

Conclusiones y Trabajo Futuro

Retos Superados

El mayor reto fue aplicar patrones de diseño reales sin sacrificar la simplicidad de la aplicación. Se logró estructurar la app en capas limpias y coherentes, manteniendo el comportamiento funcional esperado en cada pantalla. Otro desafío importante fue integrar la gestión de estado con Provider de manera ordenada, lo cual se resolvió mediante providers dedicados para autenticación, dinosaurios y comentarios.

Mejoras Futuras

- Agregar modo oscuro con temas personalizados.
- Implementar notificaciones push para actividad nueva.
- Añadir soporte offline con caché local.
- Desarrollar pruebas unitarias y de integración.
- Integrar paginación y búsqueda avanzada en la lista de dinosaurios.